Module7_lab3

October 18, 2026

[1]:
```
import pandas as pd
```

[2]:
```
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
```

[3]:
```
df=pd.read_csv("Downloads/titanic_train.csv")
```

[4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 118.9 KB
```

[5]:
```
df.head()
```

[5]:

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

```
                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th…  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

[6]:
```
df.describe()
```

[6]:
```
       PassengerId    Survived      Pclass         Age       SibSp  \
count   891.000000  891.000000  891.000000  714.000000  891.000000
mean    446.000000    0.383838    2.308642   29.699118    0.523008
std     257.353842    0.486592    0.836071   14.526497    1.102743
min       1.000000    0.000000    1.000000    0.420000    0.000000
25%     223.500000    0.000000    2.000000   20.125000    0.000000
50%     446.000000    0.000000    3.000000   28.000000    0.000000
75%     668.500000    1.000000    3.000000   38.000000    1.000000
max     891.000000    1.000000    3.000000   80.000000    8.000000

            Parch        Fare
count  891.000000  891.000000
mean     0.381594   32.204208
std      0.806057   49.693429
min      0.000000    0.000000
25%      0.000000    7.910400
50%      0.000000   14.454200
75%      0.000000   31.000000
max      6.000000  512.329200
```

[7]:
```
df.shape
```

[7]:
```
(891, 12)
```

[8]:
```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

[9]:
```
#The Age column contained 177 missing values using the df.describe(). To handle␣
↪these without distorting the data or introducing bias from extreme outliers,␣
↪missing ages were imputed using the median age instead of the average.
```

[10]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          891 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 118.9 KB
```

[11]:
```
df["Sex_encoded"] = df["Sex"].map({"male": 0, "female": 1})
```

[12]:
```
features = ["Pclass", "Sex_encoded", "Age", "Fare"]
X = df[features]
y = df["Survived"]
```

[13]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

[14]:
```
model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)
```

[14]:
```
DecisionTreeClassifier(max_depth=3, random_state=42)
```

[15]:
```
y_pred = model.predict(X_test)
test_accuracy = accuracy_score(y_test, y_pred)
```

[16]:
```
print(f"Fitted Model: {model}")
print(f"TEST Accuracy: {test_accuracy * 100:.2f}%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Fitted Model: DecisionTreeClassifier(max_depth=3, random_state=42)
TEST Accuracy: 80.60%

Confusion Matrix:
```

```
[[140  17]
 [ 35  76]]
```

The model's mistake is guessing that certain amount of men or ticket holder survived when they died. The data relies on circumstances so the model might get tripped up by real-life exceptions where more men or other ticket holders managed make it out safely.

[ ]: